

PRACTICAL TEST AUTOMATION WITH PLAYWRIGHT

Flaky Test Troubleshooting Checklist

Systematically diagnose and fix tests that pass and fail randomly.

Introduction

Flaky tests destroy trust in a suite. This checklist gives you a systematic way to find and fix the real cause instead of papering over it with retries.

Diagnosis Checklist

- Is there a `waitForTimeout` or manual sleep? Replace with assertions.
- Are you awaiting every action and assertion?
- Does the test depend on data created by another test?
- Does it assume execution order?
- Are locators stable (role/label) or brittle (CSS/index)?
- Are animations or async loads interfering?
- Does it pass alone but fail in parallel? (shared state)
- Are you asserting a user-visible outcome, not implementation detail?

Worked Example

A test fails ~1 in 5 runs. The checklist surfaces a `waitForTimeout(1000)` before clicking a button that sometimes loads slower. Replacing it with `expect(button).toBeEnabled()` then `click()` makes it pass every time.

Fix Strategy

1. Reproduce by running the test many times (`--repeat-each`).
2. Open the trace from a failing run.
3. Address the root cause from the checklist.
4. Re-run repeatedly to confirm it is stable.

Practical Exercise

Run a suspect test with `--repeat-each=10`. If any fail, use the checklist and trace to find and fix the cause, then re-run to confirm stability.

Best Practices

- Fix flakiness at the root, do not hide it with retries.
- Make tests independent and order-agnostic.
- Use assertions instead of sleeps.

Common Mistakes

- Increasing retries to mask flakiness.
- Adding sleeps "until it stops failing".
- Ignoring shared-state coupling.

INSIDE STLC PRO TIP

Run a suspect test with `--repeat-each=10` to expose flakiness on demand. A test you cannot make fail reliably is one you cannot prove you have fixed.